```sql
CREATE TABLE genres (
    genre_id INT PRIMARY KEY,
    genre_name VARCHAR(50) NOT NULL
);

CREATE TABLE movies (
    movie_id INT PRIMARY KEY,
    movie_title VARCHAR(100) NOT NULL,
    genre_id INT,
    release_year INT,
    budget_millions DECIMAL(8,2),
    revenue_millions DECIMAL(8,2),
    rating DECIMAL(3,1),
    FOREIGN KEY (genre_id) REFERENCES genres(genre_id)
);
```

```sql
INSERT INTO genres VALUES
(1, 'Action'),
(2, 'Comedy'),
(3, 'Drama'),
(4, 'Sci-Fi');

INSERT INTO movies VALUES
(101, 'Shadow Force', 1, 2022, 50.00, 150.00, 8.1),
(102, 'Funny Days', 2, 2021, 20.00, 60.00, 7.2),
(103, 'The Last Letter', 3, 2023, 30.00, 90.00, 8.4),
(104, 'Space Beyond', 4, 2024, 80.00, 240.00, 8.5),
(105, 'Final Strike', 1, 2025, 70.00, 210.00, 7.8),
(106, 'Laugh Out Loud', 2, 2022, 15.00, 45.00, 6.9),
(107, 'Broken Dreams', 3, 2021, 25.00, 75.00, 7.9),
(108, 'Galaxy Quest', 4, 2023, 60.00, 180.00, 8.2),
(109, 'Dark Mission', 1, 2024, 90.00, 270.00, 8.0),
(110, 'Happy Ending', 2, 2025, 25.00, 100.00, 7.5);
```

genre_id	genre_name
1	Action
2	Comedy
3	Drama
4	Sci-Fi

movie_id	movie_title	genre_id	release_year	budget_millions	revenue_millions	rating
101	Shadow Force	1	2022	50.00	150.00	8.1
102	Funny Days	2	2021	20.00	60.00	7.2
103	The Last Letter	3	2023	30.00	90.00	8.4
104	Space Beyond	4	2024	80.00	240.00	8.5
105	Final Strike	1	2025	70.00	210.00	7.8
106	Laugh Out Loud	2	2022	15.00	45.00	6.9
107	Broken Dreams	3	2021	25.00	75.00	7.9
108	Galaxy Quest	4	2023	60.00	180.00	8.2
109	Dark Mission	1	2024	90.00	270.00	8.0
110	Happy Ending	2	2025	25.00	100.00	7.5

1) Find high-rated movies and sort by rating

```
SELECT movie_title, release_year, rating FROM movies WHERE rating >= 8.0 ORDER BY rating DESC;
```

movie_title	release_year	rating
Space Beyond	2024	8.5
The Last Letter	2023	8.4
Galaxy Quest	2023	8.2
Shadow Force	2022	8.1
Dark Mission	2024	8.0

Purpose: Find movies rated 8.0 or higher, with the highest-rated movies first.

Insight: Identifies the best-rated movies in the dataset.

2) Count movies in each genre

```
SELECT g.genre_name, COUNT(m.movie_id) AS total_movies FROM genres g JOIN movies m ON g.genre_id = m.genre_id GROUP BY g.genre_name ORDER BY total_movies DESC;
```



genre_name	total_movies
Action	3
Comedy	3
Drama	2
Sci-Fi	2

Purpose: Calculate how many movies belong to each genre.

Insight: Shows which genres have the most movies in the sample.

3) Analyze revenue by genre

```
SELECT g.genre_name,
    SUM(m.revenue_millions) AS total_revenue,
    ROUND(AVG(m.revenue_millions), 2) AS average_revenue
FROM genres g
JOIN movies m ON g.genre_id = m.genre_id
GROUP BY g.genre_name
ORDER BY total_revenue DESC;
```

genre_name	total_revenue	average_revenue
Action	630.00	210.00
Sci-Fi	420.00	210.00
Comedy	205.00	68.33
Drama	165.00	82.50

Purpose: Compare total and average movie revenue across genres.

Insight: Reveals which genre generates the most revenue in this sample.

4) Find the highest- and lowest-revenue movies

```
SELECT movie_title, revenue_millions
FROM movies
WHERE revenue_millions = (
    SELECT MAX(revenue_millions)
    FROM movies
)
OR revenue_millions = (
    SELECT MIN(revenue_millions)
    FROM movies
);
```

movie_title	revenue_millions
Laugh Out Loud	45.00
Dark Mission	270.00

Purpose: Identify the movies with the maximum and minimum revenue.

Insight: Highlights the revenue range in the dataset.